

CS595 (Tuesday 9 AM)

CAPSTONE COURSE

San Francisco Bay University, Fremont, CA
FINAL REPORT
Term Fall 2025



Prof. Ahmed Banafa

TWOS (TravelWithOutStress)

December 9, 2025

Team Members

Umair Dada (Team Leader)	159855
Sangmin Lee	162342
Ayoyimika Ajibade	144210

Table of Contents

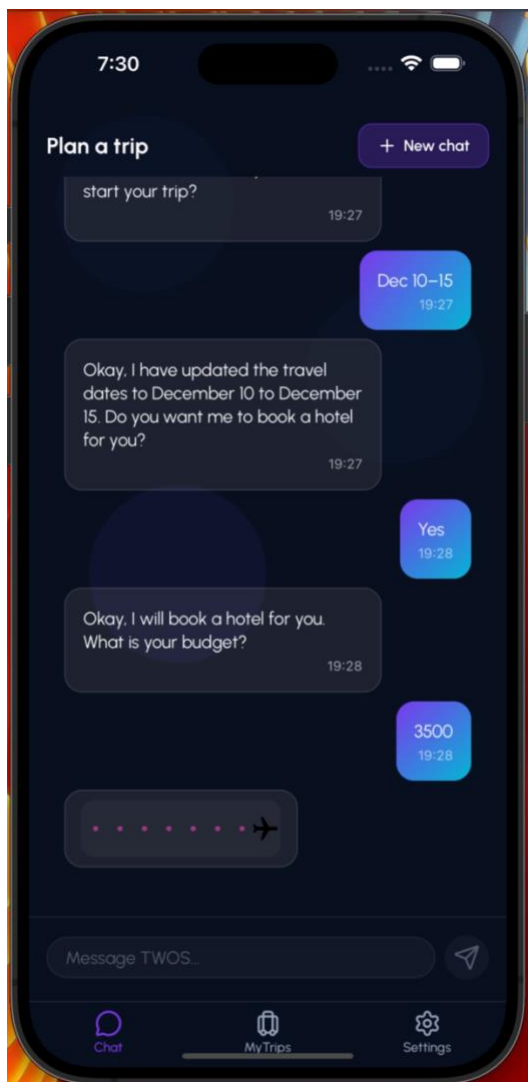
ABSTRACT	2
FRONTEND (UMAIR).....	3
2. NLP MODULE (SANGMIN).....	10
EXTERNAL APIS INTEGRATION FOR MAKING PLAN (SANGMIN).....	14
4. DATABASE AND BACKEND API MANAGEMENT (AYO).....	16
CONCLUSION.....	32
TEAM REFLECTIONS.....	33
REFERENCES.....	34

Abstract.

An AI-powered travel planner would allow people to simplify the process of organizing trips. Expedia provides part of the solution, but the main problem remains, which humans have to search for each component of trips. By combining natural language understanding with real-time data from external APIs, users can enter requirements in natural language, and the NLP module in the system parses inputs, validates, and revises details, and coordinates with the backend to search flights, hotels, car rentals, and attractions. The backend manages the NLP module and external APIs communication, and data storage, while the frontend provides a user-friendly interface for presenting results.

Frontend (Umailr)

The frontend portion of TWOS has been an unexpectedly rich and honestly kind of eye-opening process for us. At the beginning, we set out with a clear idea of creating an app that feels smooth and clean, but somewhere along the way we realized that building a mobile interface that people might actually want to use requires way more attention to detail than any of us originally thought. Working with React Native and Expo allowed us to iterate quickly and test on real devices, which created a more natural design workflow and helped us understand how the app feels in the hands of a user rather than just inside an emulator window. By the end, the frontend evolved into something that feels cohesive, functional and surprisingly close to what a real-world travel app might look like.

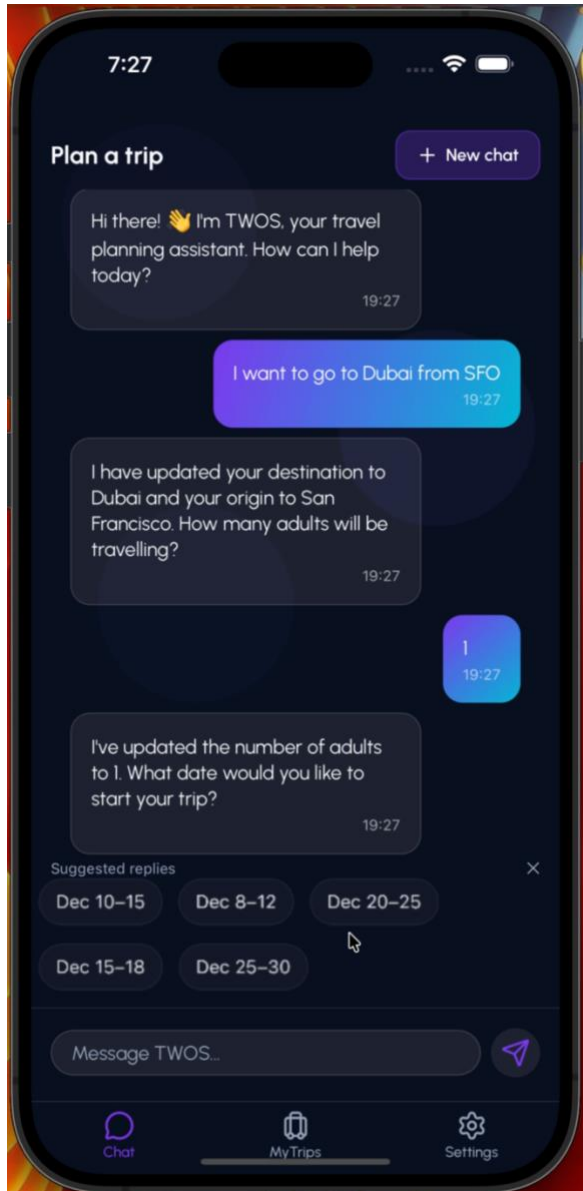


1 User Interface and Experience

We focused a lot on the overall theme of TWOS. Since the name itself suggests a stress-free experience, we tried to reflect that idea through gradients, soft surfaces, balanced spacing and a calm color palette. Some screens took several attempts to get right because there were moments when the spacing felt off or the shadows were too sharp or the buttons didn't match the rest of the app. These small imperfections really stood out during testing, and, honestly, they pushed us to redo sections more often than we expected. Over time, the design matured into something consistent. Every screen, whether it is the home interface or the settings page, now follows the same visual language.

Figure 1.1 shows the home layout that introduces the main navigation and overall design style.

2 Chat Assistant Interface



The chat interface became the centerpiece of the frontend. We wanted this area to feel as natural and conversational as possible, similar to how a user might talk to a real assistant. This meant reworking message bubbles, adjusting the alignment of text blocks, experimenting with gradient bubbles and rewriting the spacing logic until everything began to flow more comfortably.

We added quick-action chips because we noticed during testing that typing repeated things felt tiring. And we included inline previews of trip suggestions directly inside the chat. It feels practical and gives the user a sense that they do not have to jump around the app just to understand what the assistant is offering.

Figure 1.2 shows a preview card appearing inside the conversation.

3 Navigation and Application Structure

For the final version of TWOS, the navigation structure includes three primary tabs: Chat, My Trips and Settings. Earlier in development, we experimented with more screens, including a separate packages section, but it eventually felt unnecessary and sometimes even confusing.

Consolidating everything into a simpler three-tab layout made the app easier to understand and more comfortable to navigate.

Small adjustments had a surprisingly big impact. Things like icon sizing, the distance between the labels and the tab bar, and even the timing of transitions all influenced how the app feels during real use. These are subtle details, but we noticed that when they were slightly off, the app felt less professional. Once they were corrected, everything suddenly felt more grounded.

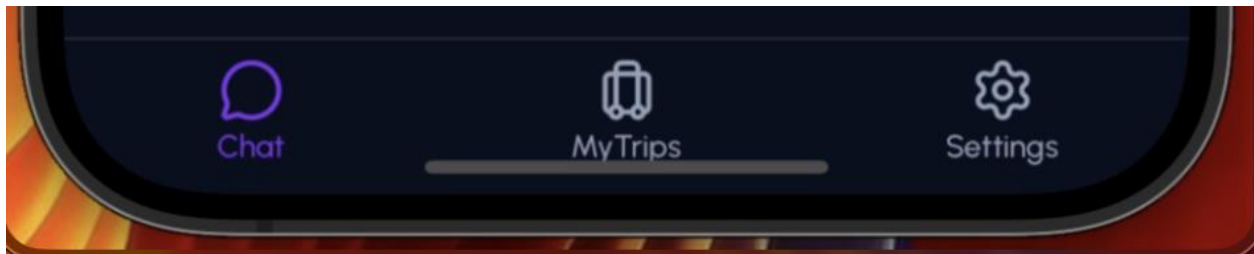


Figure 1.4 contains a view of the final tab bar arrangement

4 Trip Preview and Checkout Flow

Since we no longer use a separate packages screen, the trip previews appear directly after a user taps a suggested option inside the chat. This flow keeps the user anchored in one place rather than shuffling them between multiple sections. The preview screens take inspiration from well-known booking platforms, with structured information, pricing details and room or flight summaries.

The checkout flow is simulated for now but designed in a way that shows how a real payment system would eventually feel. Users can choose a payment option, confirm their selection and then return to the chat with a sense of completion. Although the payments are mocked, the UI resembles an actual checkout experience closely enough that testers understood the intention without needing explanation

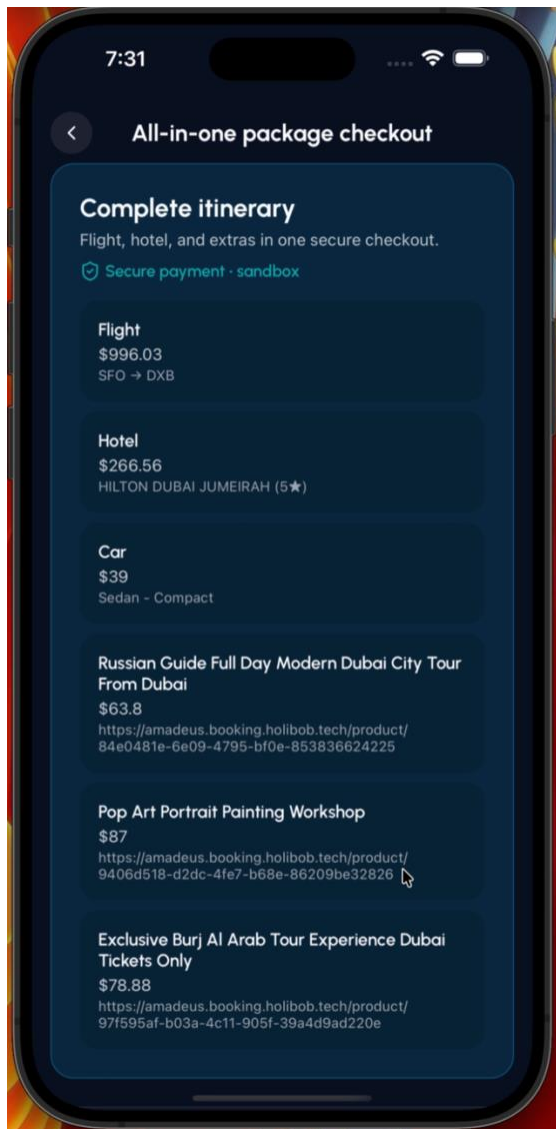


Figure 1.5 displays a trip preview page.

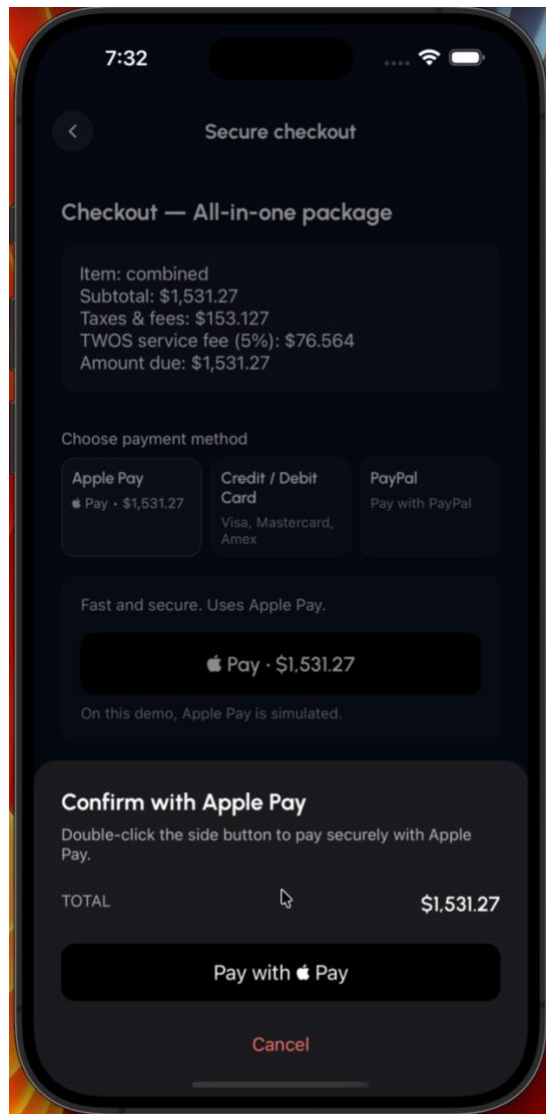
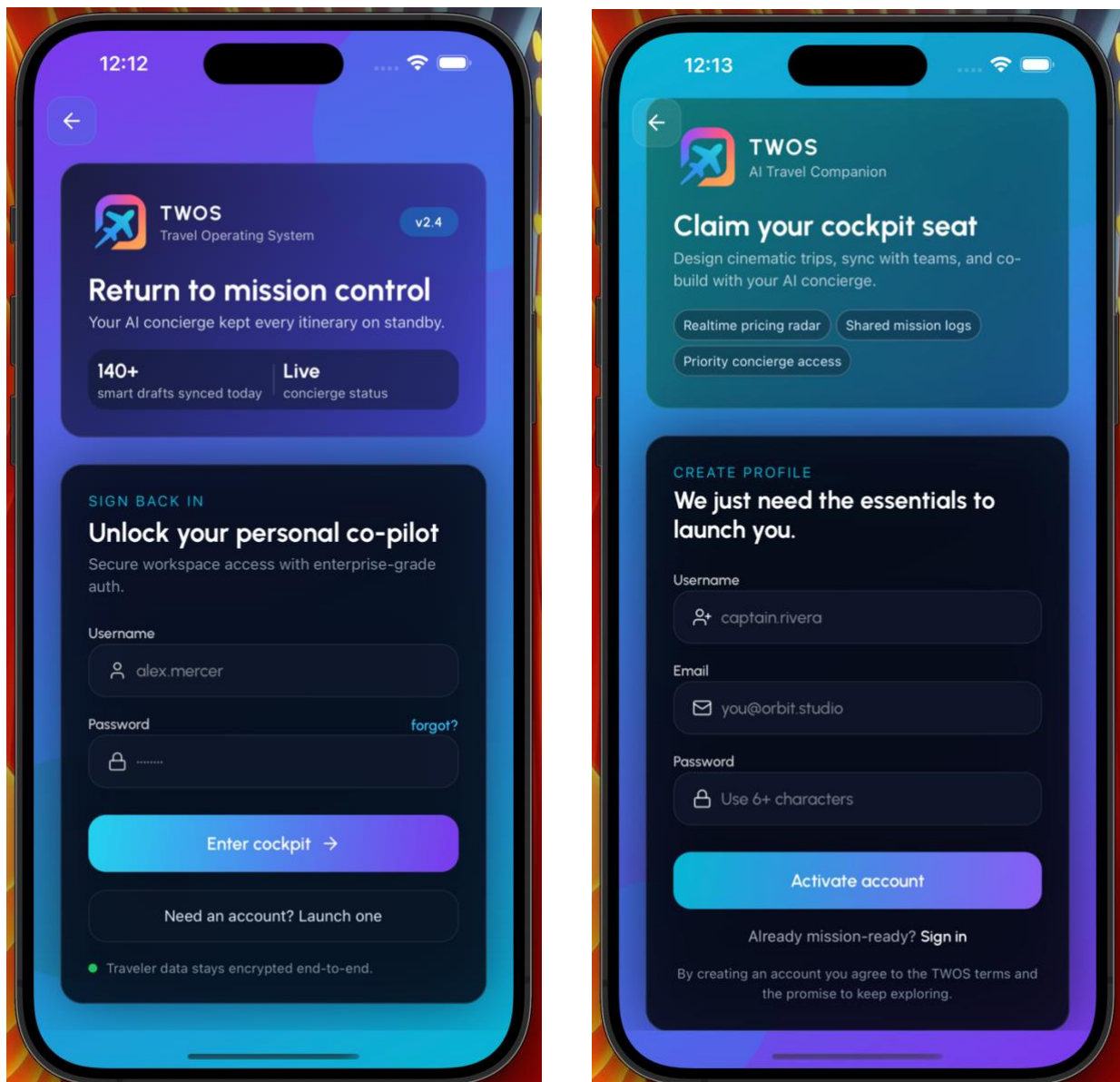


Figure 1.6 shows the mock checkout interface.

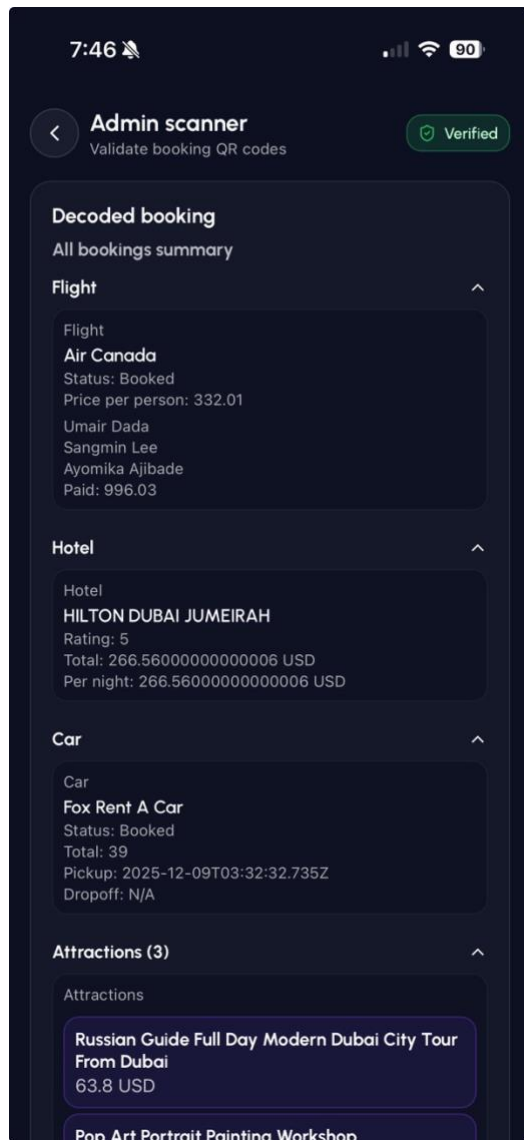
5 Authentication Screens

The login and signup screens were developed to match the app's visual theme. We applied gradients, spacing patterns, and button styles similar to the rest of TWOS so the experience feels unified. Even though the authentication logic currently works with placeholder responses, the screens are structured in a way that they can easily connect to a real backend later.



Figures 1.7 and 1.8 illustrate the login and signup layouts

6 Admin Scanner



The Admin Scanner is a simple but meaningful addition. It allows scanning booking or trip reference codes and then displaying the associated data. While the data itself is still simulated, the scanner UI functions correctly and previews the direction of future administrative tools. We tried to design it in a minimal and straightforward fashion since it is meant for internal use rather than end users.

Figure 1.9 shows the Admin Scanner in action.

7 Component Architecture

Throughout development, we built a set of reusable components that appear across multiple screens. Examples include the message bubble component, the gradient button, the chip element and trip cards. Building reusable components forced us to think more carefully about structure and styling. There were days when we rewrote a component several times just to make sure it felt adaptable enough for multiple use cases. This approach also made the final interface more consistent because adjusting a single component automatically updated every screen that used it.

8 Backend Integration Readiness

Although much of the current data is mocked, the frontend is designed to integrate cleanly with the backend as soon as the API endpoints are connected. We used Axios with environment variables and prepared React Query hooks that handle caching, loading states and request errors. We also validated screens using placeholder responses to make sure that unexpected data shapes would not break the UI. The application should transition smoothly into a fully connected state when the backend is ready.

9 Deployment on Expo Go

Deploying TWOS through Expo Go felt like an important step for us. It made the app accessible to classmates, faculty and anyone else who wanted to test it quickly. Seeing the app run on actual devices instead of only inside an emulator gave us a clearer understanding of real-world usability. The deployment also made feedback easier because users could simply scan a QR code and interact with the application without needing to install anything complicated.

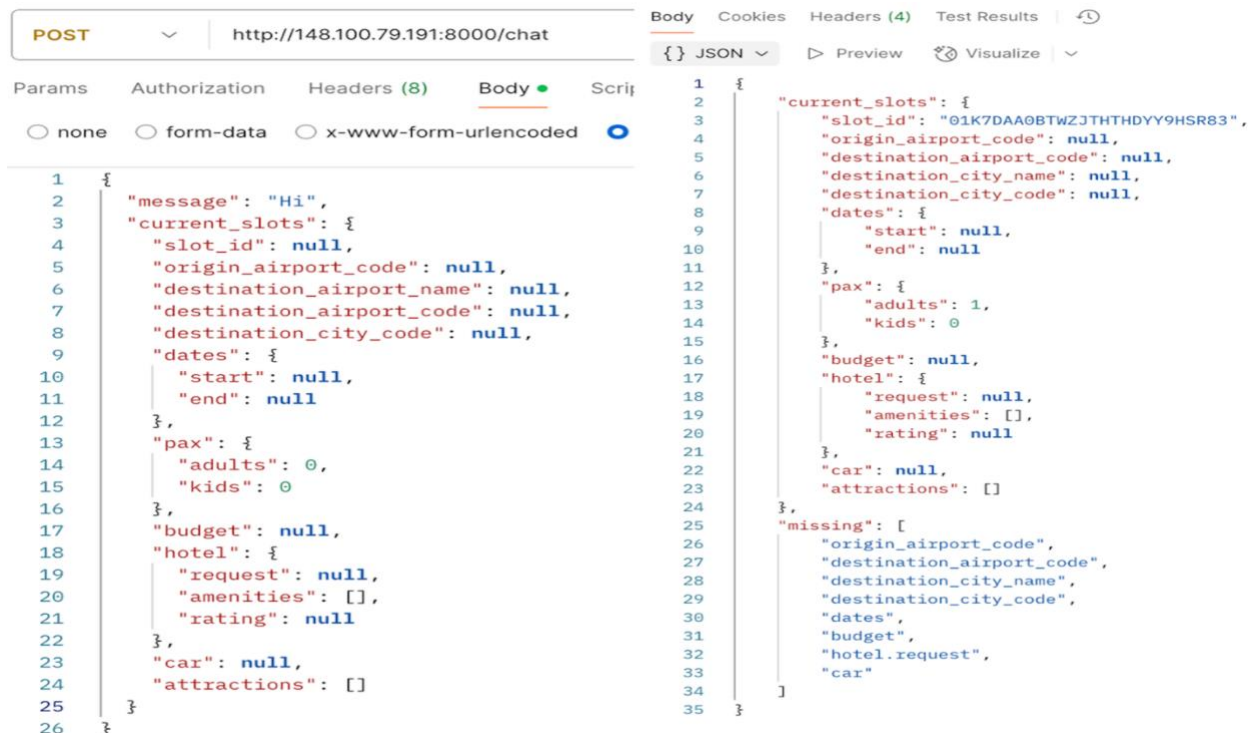
Group ID	Branch	Runtime	Commit	Created by	Created at	Updated at
ab97e009	production	1.0.0	740e17f*	umalrdaaa	Dec 5, 2025 11:58 PM	Dec 5, 2025 11:58 PM
Platform-specific updates						
Platform	ID	Fingerprint	Downloads	Average size	Known launches	Known crashes
Android	a403fba8	84f5dbc	None	None	None	None
iOS	deb0d2dd	823b0b0	5	None	None	None



Figure 2.0 shows the Expo Go deployment with the QR code used for testing

2. NLP Module (Sangmin)

For development efficiency, we separate the backend and the NLP module parts. Currently, the NLP module provides two services—**chat** and **package**. The **chat** service interacts with Frontend, and it has three major functionalities. First, it generates a unique slot ID if a slot ID is empty. It implies that the user starts a new chat. This slot ID is used to track slots during and after the chat. Second, it handles the user's natural language and fills/updates the slots. This handling includes parsing the user's message and replying to the user to ask for missing information. Third, it provides travel plans. This feature was in the search service, but we have successfully integrated it into the chat service, and this is a significant improvement we made to optimize the application. The following are samples of the chat service.



The screenshot displays a REST client interface with a POST request to `http://148.100.79.191:8000/chat`. The request body is a JSON object with the following structure:

```
1 {
2   "message": "Hi",
3   "current_slots": {
4     "slot_id": null,
5     "origin_airport_code": null,
6     "destination_airport_name": null,
7     "destination_airport_code": null,
8     "destination_city_code": null,
9     "dates": {
10      "start": null,
11      "end": null
12    },
13    "pax": {
14      "adults": 0,
15      "kids": 0
16    },
17    "budget": null,
18    "hotel": {
19      "request": null,
20      "amenities": [],
21      "rating": null
22    },
23    "car": null,
24    "attractions": []
25  }
26 }
```

The response body is a JSON object with the following structure:

```
1 {
2   "current_slots": {
3     "slot_id": "01K7DAA0BTWZJTHTHDYY9HSR83",
4     "origin_airport_code": null,
5     "destination_airport_code": null,
6     "destination_city_name": null,
7     "destination_city_code": null,
8     "dates": {
9       "start": null,
10      "end": null
11    },
12    "pax": {
13      "adults": 1,
14      "kids": 0
15    },
16    "budget": null,
17    "hotel": {
18      "request": null,
19      "amenities": [],
20      "rating": null
21    },
22    "car": null,
23    "attractions": []
24  },
25   "missing": [
26     "origin_airport_code",
27     "destination_airport_code",
28     "destination_city_name",
29     "destination_city_code",
30     "dates",
31     "budget",
32     "hotel.request",
33     "car"
34   ]
35 }
```

NLP Module Figure 1. Initial request and response when the user starts a new chat

POST http://148.100.79.191:8000/chat

Body

```

1 {
2   "message": "I am planning a family trip from SF to Seoul. There are 4 people, 2 adults and 2 kids. From Nov 10 to Nov 25. I need a rental car during the trip. I don't need a hotel. My budget is $5k.",
3   "current_slots": {
4     "slot_id": "01K7DAA0BTWZJTHDYY9HSR83",
5     "origin_airport_code": null,
6     "destination_airport_code": null,
7     "destination_city_name": null,
8     "destination_city_code": null,
9     "dates": {
10      "start": null,
11      "end": null
12    },
13    "pax": {
14      "adults": 1,
15      "kids": 0
16    },
17    "budget": null,
18    "hotel": {
19      "request": null,
20      "amenities": [],
21      "rating": null
22    },
23    "car": null,
24    "attractions": []
25  },
26  "missing": [
27    "origin_airport_code",
28    "destination_airport_code",
29    "destination_city_name",
30    "destination_city_code",
31    "dates",
32    "budget",
33    "hotel.request",
34    "car"
35  ]
36 }

```

JSON

```

1 {
2   "current_slots": {
3     "slot_id": "01K7DAA0BTWZJTHDYY9HSR83",
4     "origin_airport_code": "SFO",
5     "destination_airport_code": "ICN",
6     "destination_city_name": "Seoul",
7     "destination_city_code": "SEL",
8     "dates": {
9       "start": "2025-11-10",
10      "end": "2025-11-25"
11    },
12    "pax": {
13      "adults": 2,
14      "kids": 2
15    },
16    "budget": 5000.0,
17    "hotel": {
18      "request": false,
19      "amenities": [],
20      "rating": null
21    },
22    "car": true,
23    "attractions": []
24  },
25  "missing": []
26 }

```

NLP Module Figure 2. Parsing the user's natural language and filling out the slots

POST http://148.100.79.191:8000/chat

Body

```

1 {
2   "message": "I want to leave from the San Jose instead of SF",
3   "current_slots": {
4     "slot_id": "01K78884QKMEKEDZ4Z45K46CD",
5     "origin_airport_code": "SFO",
6     "destination_airport_code": "ICN",
7     "destination_city_name": "Seoul",
8     "destination_city_code": "SEL",
9     "dates": {
10      "start": "2025-11-10",
11      "end": "2025-11-25"
12    },
13    "pax": {
14      "adults": 2,
15      "kids": 2
16    },
17    "budget": 5000.0,
18    "hotel": {
19      "request": false,
20      "amenities": [],
21      "rating": null
22    },
23    "car": true,
24    "attractions": []
25  },
26  "missing": []
27 }

```

JSON

```

1 {
2   "current_slots": {
3     "slot_id": "01K78884QKMEKEDZ4Z45K46CD",
4     "origin_airport_code": "SJC",
5     "destination_airport_code": "ICN",
6     "destination_city_name": "Seoul",
7     "destination_city_code": "SEL",
8     "dates": {
9       "start": "2025-11-10",
10      "end": "2025-11-25"
11    },
12    "pax": {
13      "adults": 2,
14      "kids": 2
15    },
16    "budget": 5000.0,
17    "hotel": {
18      "request": false,
19      "amenities": [],
20      "rating": null
21    },
22    "car": true,
23    "attractions": []
24  },
25  "missing": []
26 }

```

NLP Module Figure 3. Revise the slot field based on the user's request

```

{
  "message": "I do need hotel",
  "current_slots": {
    "slot_id": "01K78884QKMEKEDZ4Z45K46CD",
    "origin_airport_code": "SFO",
    "destination_airport_code": "ICN",
    "destination_city_name": "Seoul",
    "destination_city_code": "SEL",
    "dates": {
      "start": "2025-12-10",
      "end": "2025-12-15"
    },
    "pax": {
      "adults": 2,
      "kids": 2
    },
    "budget": 5000.0,
    "hotel": {
      "request": true,
      "amenities": [],
      "rating": null
    },
    "car": true,
    "attractions": []
  },
  "missing": []
}

"flight": {
  "airline": "UA",
  "price_per_person": 4232.32,
  "currency": "USD",
  "departure_time": "2025-12-10T10:30:00",
  "arrival_time": "2025-12-11T16:15:00",
  "flight_type": null,
  "link": "https://www.amadeus.com"
},
"hotel": {
  "name": "Marriott Executive Apartments Yeouido Park Centre Seoul",
  "rating": null,
  "total_price": 1501.04493,
  "price_per_night": 300.208986,
  "currency": "USD",
  "link": "https://www.amadeus.com"
},
"car": {
  "company": "Budget",
  "car_type": "SUV - Standard",
  "price_per_day": 75.5,
  "currency": "USD",
  "link": "https://www.budget.com/en/vehicles/us"
},
"attractions": [
  {
    "name": "SEOUL Mobile Public Transport Card for Subway and Bus",
    "price": 13.0,
    "currency": "USD",
    "link": "https://amadeus.booking.holibob.tech/product/9e9df5d9"
  }
]

```

NLP Module Figure 4. Chat service call APIs, organize them, and send the travel plan to FE.

The **package** service provides pre-built travel plans. It provides 3 different randomly generated plans based on the budget (under \$2000, \$2000 - \$5000, over \$5000). At the production level, to activate this feature, users need to allow tracking of their location information. However, in the development environment, we set the departure location to San Francisco (SFO), and the destinations were categorized based on the budget:

1. Under \$2000: Nearby cities within California (e.g., Los Angeles, Las Vegas, Seattle, San Diego, and more)
2. \$2000 - \$5000: Major cities in the eastern and central states (e.g., New York, Chicago, Boston, Miami, Washington D.C., and more)
3. Over \$5000: Popular international cities (e.g., London, Tokyo, Seoul, Paris, and more)

The following are samples of the package service.

```

"by_tier": {
  "under_2000": {
    "plan_id": "01KA9EDSKBH98N0AWN6MPBE4EW",
    "slot_id": "01KA9EDSKB8NGJFK9MRB62043V",
    "flight": {
      "airline": "F9",
      "price_per_person": 525.9,
      "currency": "USD",
      "departure_time": "2025-11-21T06:00:00",
      "arrival_time": "2025-11-21T07:41:00",
      "flight_type": null,
      "link": "https://www.amadeus.com"
    },
    "hotel": {
      "name": "SPRINGHILL STES NORTH MARRIOTT",
      "rating": 2,
      "total_price": 404.54,
      "price_per_night": 202.27,
      "currency": "USD",
      "link": "https://www.amadeus.com"
    },
    "car": {
      "company": "Fox Rent A Car",
      "car_type": "Sedan - Compact",
      "price_per_day": 39.0,
      "currency": "USD",
      "link": "https://www.foxrentacar.com/en/rental-cars.html"
    },
    "attractions": [
      {
        "name": "Madame Tussauds Las Vegas admission tickets with Marvel 4D",
        "price": 40.0,
        "currency": "USD",
        "link": "https://amadeus.booking.holibob.tech/product/3057494f-f266-4c76-b1e2-c579713922d3"
      }
    ]
  },

```

NLP Module Figure 5. Package service – Under \$2000

```

"between_2000_5000": {
  "plan_id": "01KA9EF205AN7W5YCN0ZH19CYT",
  "slot_id": "01KA9EF205KSNDK4ARKN90TD1V",
  "flight": {
    "airline": "F9",
    "price_per_person": 1400.97,
    "currency": "USD",
    "departure_time": "2025-11-22T22:17:00",
    "arrival_time": "2025-11-23T11:33:00",
    "flight_type": null,
    "link": "https://www.amadeus.com"
  },
  "hotel": {
    "name": "Best Western Plus Plaza Hotel",
    "rating": 3,
    "total_price": 496.08,
    "price_per_night": 124.02,
    "currency": "USD",
    "link": "https://www.amadeus.com"
  }
},

```

NLP Module Figure 6. Package service – \$2000 - \$5000

External APIs Integration for Making Plan (Sangmin)

The 3 external APIs and 1 mock API are integrated into the chat service. We used the Amadeus API for flights, hotels, and attractions, since it provides free and high-quality data in a test environment. For a rental car, there is no free-tier API, so we make a mock API. In utilizing data from external APIs, we set basic principles to handle and make an optimized travel plan.

1. Request the data.
2. Receive data from the APIs.
3. Filter and/or sort data based on the users' requests.

The reason for principles 1 to 3 is to select options that best match the users' budgets and preferences, although it spends more resources.

4. Hotels & Cars:
 - a. If the "request" field is false, then skip the related API calls to save resources.
5. Attractions:
 - a. Although the attractions field is empty, call the APIs for a recommendation.
6. Chosen data should be stored in local storage (guest user) or DB (login user)

The reason for principles 4 to 6 is to optimize the resources and save data. By setting and following these principles, we are expecting not only better resource management, but also better trip plans based on the user's requests. The following are examples of how these principles are worked and applied. The following are API logs.



API Figure 1. Log to check/debug that APIs are working properly and following the principles 1

```
✓ CAN SEARCH ATTRACTIONS: True ✓  
{'name': '9/11 Ground Zero Tour', 'description': '  
{'name': 'The National September 11 Memorial Museu  
SD', 'link': 'N/A'}  
{'name': 'Brooklyn Bridge', 'description': '', 'ra  
📊 Attraction search returned 3 results  
✓ CAN SEARCH CAR: True ✓  
🚗 Searching for car...  
🚗 Found company='Fox Rent A Car' car_type='Sedan - Compact' price_per_day=39.0  
ntacar.com/en/rental-cars.html' car.  
🚗 Car rental for 5 days at 39.0/day = $195.0  
💰 The total price for the travel plan is: $3777.10
```

API Figure 2. Log to check/debug that APIs are working properly and following the principles 2

4. Database and Backend API Management (Ayo)

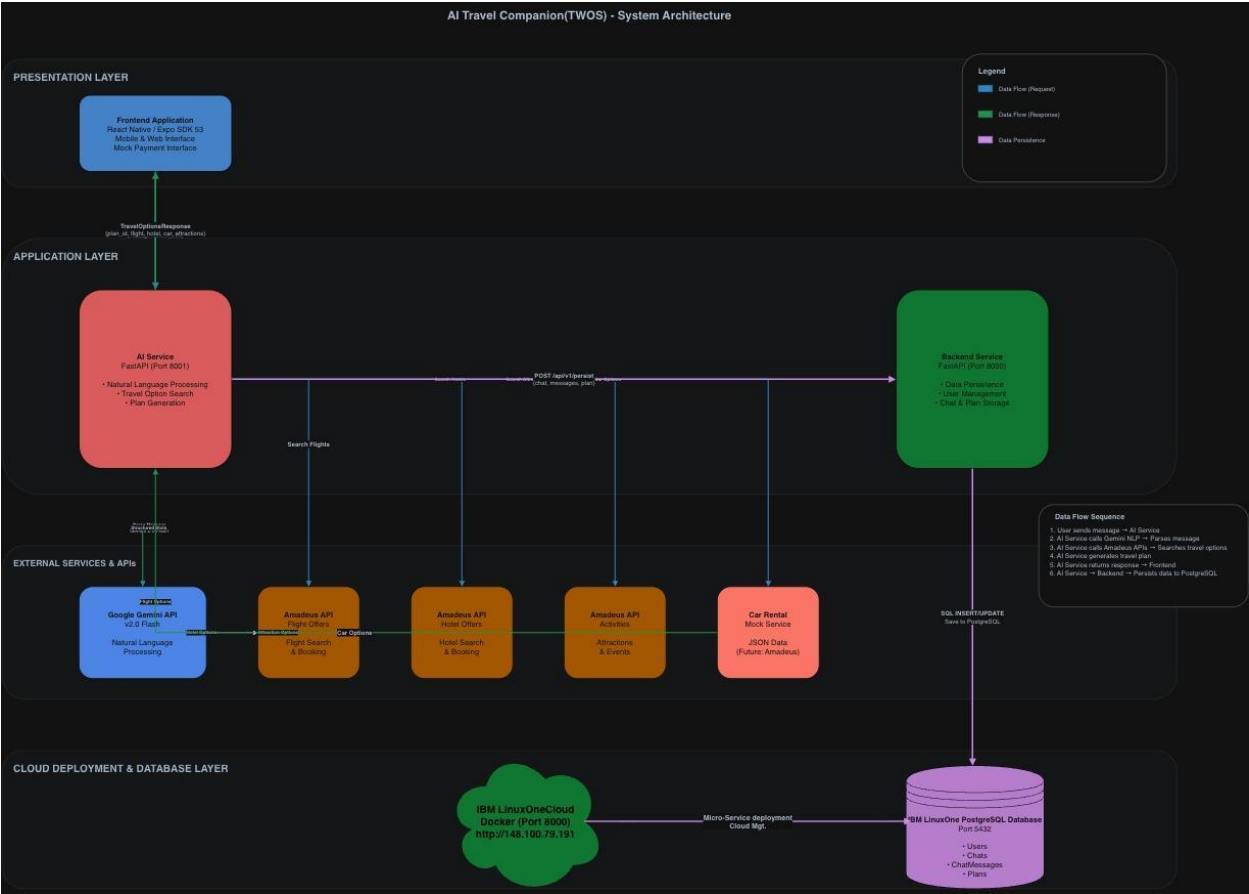


Figure 1 - High-Level System Architecture

Description: Diagram showing the overall system architecture with frontend, AI, backend, database, and external service interactions.

Key Components and Their Interactions

1. Client Layer

Mobile Application (React Native with Expo)

The mobile application serves as the primary user interface for the system. Built with React Native and Expo SDK 53, the application provides:

- Chats Interface: Natural language chat interface for travel planning
- Authentication: Secure user registration, login, and session management
- Plan Management: View, compare, and manage generated travel plans
- Chat History: Access to chat history and saved plans
- RESTful API Integration: Direct HTTPS communication with backend services

The application communicates exclusively through synchronous HTTP/REST API calls, ensuring compatibility with standard mobile networking environments.

2. AI Service (Entry Point Service)

Technology: FastAPI application running in Docker container (**Port 8000**)

Purpose: The AI Service serves as the primary entry point for all client interactions and handles the system's intelligent processing layer. This service is responsible for:

Natural Language Processing (NLP):

- Receives user messages in natural language format
- Utilizes *Google Gemini 2.0 Flash API* for intent extraction and entity recognition
- Parses unstructured user input into structured travel parameters (*origin, destination, dates, budget, preferences*)
- Maintains chat context using slot-based tracking (*ULID slot_id*)

Travel Data Aggregation:

- Integrates with **Amadeus** for Developers APIs for real-time travel information:
 - *Flight Offers API*: Searches and retrieves flight options based on origin, destination, and dates
 - *Hotel Offers API*: Queries hotel availability and pricing
 - *Activities API*: Discovers local attractions and activities

- *Mock Car Rental Service*: Provides car rental recommendations (prepared for future Amadeus Car API integration)
- Aggregates travel options into comprehensive travel plans
- Applies filtering and selection algorithms to identify optimal options

Chat Management:

- Generates unique session identifiers (**slot_id**) using ULID for session continuity
- Merges new user input with existing conversation state
- Determines information completeness and prompts for missing required parameters
- Generates contextual, natural language responses to guide users through the planning process

Plan Generation:

- Automatically generates travel plans when all required information is collected
- Creates structured JSON responses containing flight, hotel, car rental, and attraction recommendations
- Calculates total pricing and provides budget analysis
- Returns immediate responses to users while asynchronously persisting data

Data Persistence Coordination:

- Makes non-blocking HTTP requests to the Backend Service to persist chat conversations, messages, and generated plans
- Ensures data durability without impacting user response times

3. Backend Service (Persistence & Management Layer)

Technology: FastAPI application running in Docker container (**Port 8001**)

Purpose: The Backend Service handles all data persistence, user management, and administrative operations:

Authentication & Authorization:

- *User Registration:* Creates new user accounts with username, email, and encrypted password storage
- *Login/Logout:* JWT-based authentication with access tokens (8-hour expiration) and refresh tokens (30-day expiration)
- *Token Management:* Implements token blacklisting for secure logout functionality
- *Password Management:* Supports password reset with temporary password generation and secure password change functionality
- *Optional Authentication:* Supports both authenticated and anonymous user sessions

User Management:

- Profile management (retrieve and update user information)
- User preferences storage (flexible JSON-based preference system)
- Last login tracking and account status management

Chat Management:

- Chat session creation and retrieval (linked by slot_id for anonymous users or user_id for authenticated users)
- Chat status management (draft, booked, confirmed, cancelled)
- Chat ownership and access control
- Chat deletion with cascade deletion of associated plans and messages
- Support for shareable chat links via unique share_code

Plan Management:

- Travel plan storage and retrieval
- Plan association with chat sessions
- Plan deletion and management
- Public plan access (unauthenticated plan viewing via plan_id)

Message Persistence:

- Stores both user messages and AI-generated responses
- Maintains chat history with role-based message classification (user/bot)
- Links messages to chat sessions via `chat_id` and `slot_id`
- Preserves AI response metadata and structured data

Database Interaction:

- Direct SQLAlchemy/SQLModel ORM interaction with PostgreSQL
- Repository pattern for data access abstraction
- Service layer for business logic encapsulation
- Transaction management and data integrity enforcement

4. Data Layer

PostgreSQL Database (IBM Cloud Databases or Self-Hosted)

The system employs PostgreSQL 13+ as the primary persistent storage solution:

Schema Overview:

- `users`: User accounts, authentication credentials, and profile information
- `user_preferences`: Flexible JSON-based user preference storage
- `token_blacklist`: JWT token invalidation tracking (supports secure logout)
- `chats`: Travel chat sessions with comprehensive travel parameters
- `chat_messages`: Individual messages within conversations (user and bot messages)
- `plans`: Generated travel plans with flight, hotel, car, and attraction data stored as JSONB

Design Characteristics:

- UUID primary keys for all entities
- Timestamps (*`created_time`, `updated_time`*) for audit trails
- Foreign key relationships with appropriate cascade behaviors
- Indexes on frequently queried fields (*`user_id`, `chat_id`, `slot_id`, `share_code`*)
- JSONB fields for flexible schema-less data storage (*`travel_options`, `preferences`, `metadata`*)

Data Access Pattern:

- All persistent data operations go through the Backend Service
- Direct database access is not exposed to clients or the AI Service
- The repository pattern ensures consistent data access patterns
- The service layer enforces business logic and validation rules

5. External Services & APIs**AI Provider - Google Gemini 2.0 Flash:**

- Purpose: Natural language understanding and parsing
- Integration: REST API calls with JSON mode enabled
- Authentication: API key-based authentication
- Capabilities: Extracts structured travel parameters from conversational input, generates natural language responses

Travel Data Provider - Amadeus for Developers:

- Purpose: Real-time travel data aggregation
- Integration: OAuth2 client credentials flow with REST API calls
- Endpoints:
 - *Flight Offers API*: Comprehensive flight search and pricing
 - *Hotel Offers API*: Hotel availability and pricing information
 - *Activities API*: Local attractions and activities discovery
- Authentication: OAuth2 client credentials (API key and secret)
- Data Quality: Live, real-time data from travel industry providers

Future Integration - Car Rental API:

- Currently uses a mock data service for car rental recommendations
- Architecture prepared for Amadeus Car Rental API integration

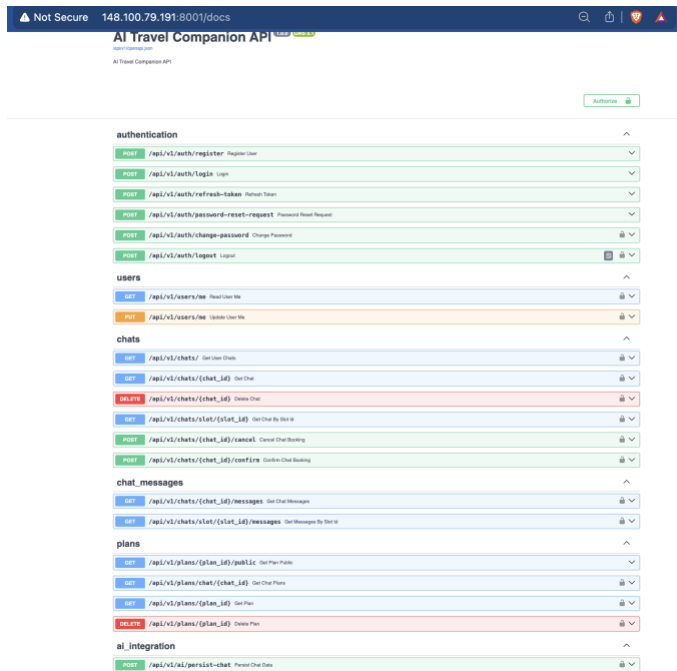


Figure 2 - API Route Structure

Description: Visual representation of the API routes and their HTTP methods.

Example:

Auth & User Mgt: POST /auth/login, GET /users/me.

Chat & Plans Mgt:

GET /plan/{plan_id}/,

GET /chats,

DELETE/chats/{chat_id}..

AI Integration: POST ai/chats/



Figure 3 - Database Schema and Models

Description: ERD

showing tables and their relationships. Robust, relational, JSONB support for flexible schema (flight/hotel data). The stated requirement.

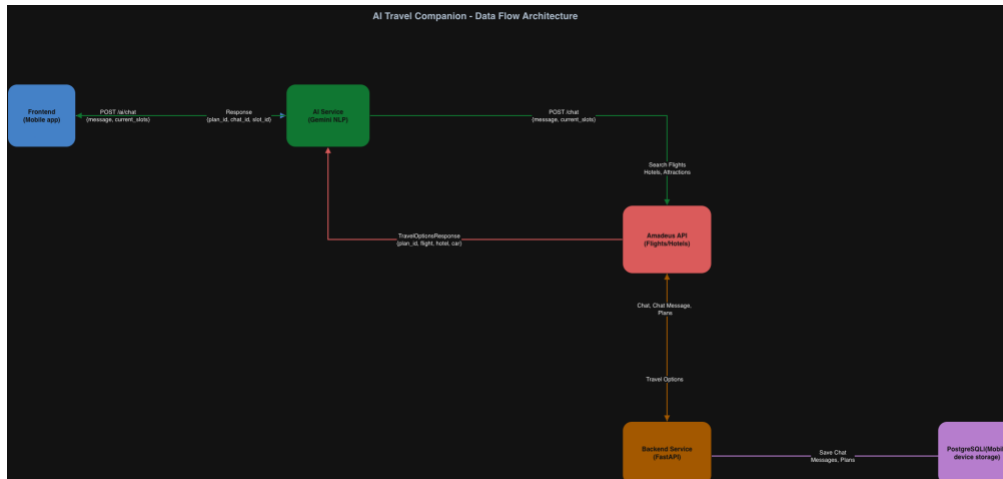


Figure 4 - TWOS

DataFlow

Description:

Example of data flow with the samples of request and response data.

1. User sends message → AI Service
2. AI Service calls Gemini NLP → Parses message
3. AI Service calls Amadeus APIs → Searches travel options
4. AI Service generates travel plan
5. AI Service returns response → Frontend
6. AI Service → Backend → Persists data to PostgreSQL

Full User Authentication & Security Flow With JWT, with full security measures Implemented

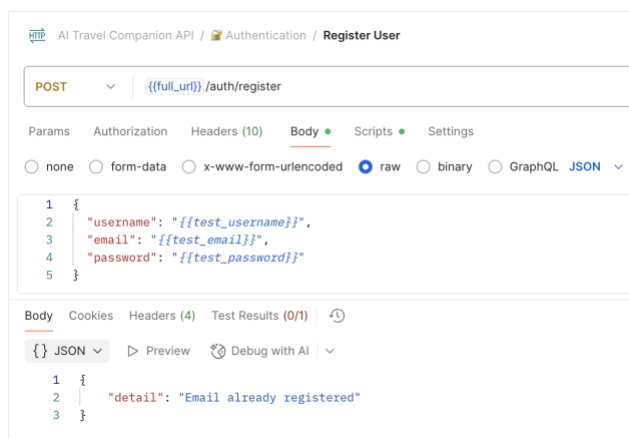


Figure 5 - User Register Flow

POST /api/v1/auth/register - User registration (requires username, email, and password only)



POST	/api/v1/auth/login	-	Uses email/username and password to generate access and refresh tokens
-------------	---------------------------	---	--



POST /api/v1/auth/logout - Server-side logout tracking for audit purposes(frontend will handle logout by removing the user token from the browser session)



POST /api/v1/auth/refresh -Access tokens expire in 8 hours, refresh tokens in 30 days

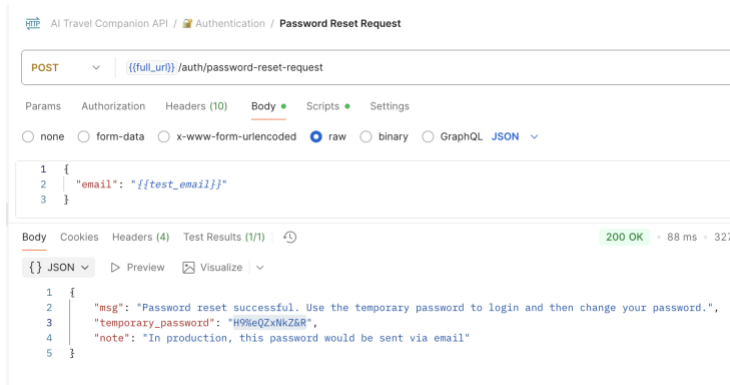


Figure 9 - User Password Reset Flow

POST /api/v1/auth/password-reset-request - Generates temporary passwords for secure password recovery

Password Reset Flow(Production)

1. User requests a password reset via email
2. The system generates a temporary password
3. User logs in with a temporary password to get an access token
4. User changes password using the change-password endpoint with an access token

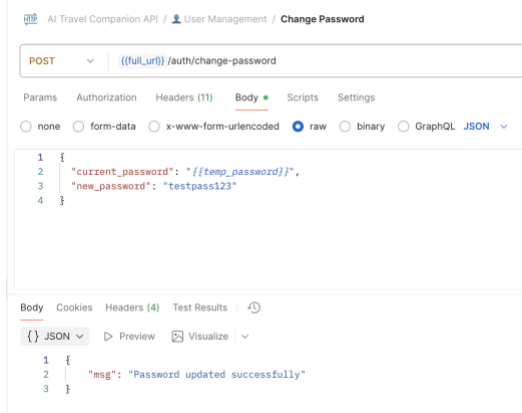


Figure 10 - User Password Change Flow

POST /api/v1/auth/change-password - Allows users to change passwords with current password verification

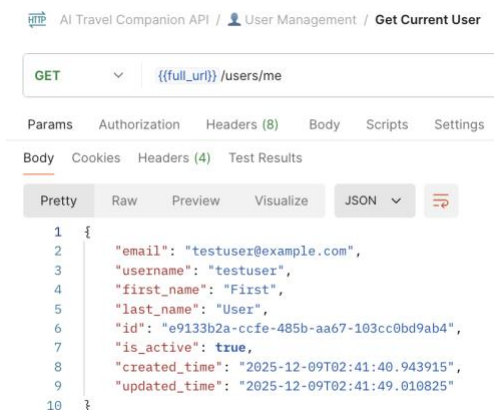
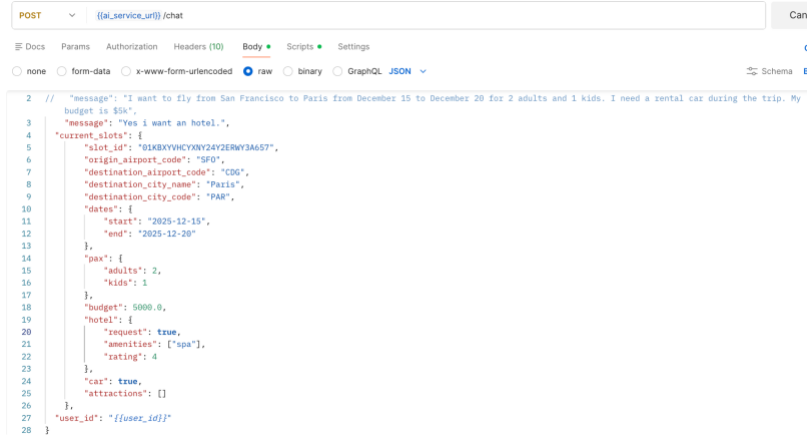


Figure 11 - Verify User is authenticated

GET /api/v1/users/me - Get current user profile

Full Chats, Chat messages, and Plans Flow, with optional authentication measures implemented



A screenshot of a REST client interface showing a POST request to the endpoint `[[{url_service_url}]]/chat`. The request body is a JSON object containing travel details: a message, current slots, origin/destination airport codes, city names, dates, passenger count, budget, hotel request, car request, and attractions. The user ID is set to `{{user_id}}`.

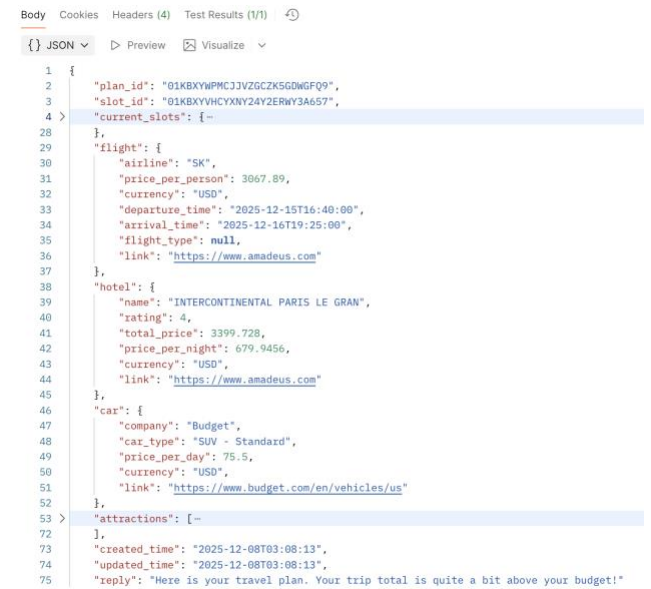
```
2 // "message": "I want to fly from San Francisco to Paris from December 15 to December 20 for 2 adults and 1 kids. I need a rental car during the trip. My budget is $5k",
3 "message": "Yes I want a hotel.",
4 "current_slots": {
5   "slot_id": "01KBXYVHCYXNY24Y2ERWY3A657",
6   "origin_airport_code": "SFO",
7   "destination_airport_code": "CDG",
8   "destination_city_name": "Paris",
9   "destination_city_code": "PAR",
10  "dates": {
11    "start": "2025-12-15",
12    "end": "2025-12-20"
13  },
14  "pax": {
15    "adults": 2,
16    "kids": 1
17  },
18  "budget": 5000.0,
19  "hotel": {
20    "request": true,
21    "amenities": ["spa"],
22    "rating": 4
23  },
24  "car": true,
25  "attractions": []
26 },
27 "user_id": "{{user_id}}"
28 }
```

Figure 12 - Chat sample request

POST chat/ - POST user messages to return plans with flight, hotel, attractions, and cars, and accessible to both unauthenticated and authenticated users

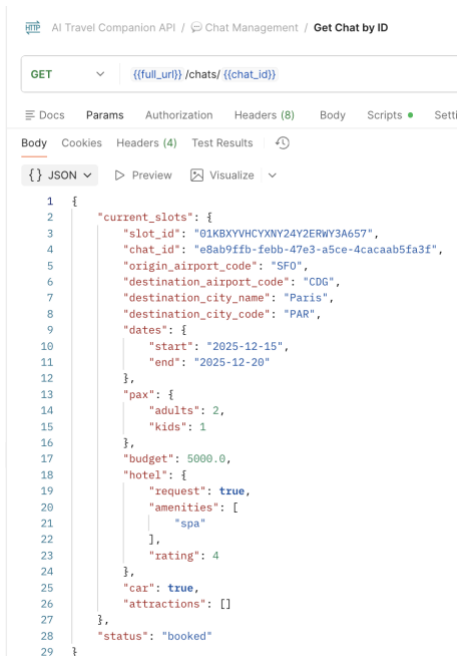
Figure 13 - Chat sample response

POST chat/ - POST user messages to return plans with flight, hotel, attractions, and cars, and accessible to both unauthenticated and authenticated users



A screenshot of a REST client interface showing the JSON response to the POST chat request. The response includes plan details, flight information, hotel details, car details, and attractions.

```
1 {
2   "plan_id": "01KBXYVHCYXNY24Y2ERWY3A657",
3   "slot_id": "01KBXYVHCYXNY24Y2ERWY3A657",
4   "current_slots": {
5     "flight": {
6       "airline": "SK",
7       "price_per_person": 3067.89,
8       "currency": "USD",
9       "departure_time": "2025-12-15T16:40:00",
10      "arrival_time": "2025-12-16T19:25:00",
11      "flight_type": null,
12      "link": "https://www.amadeus.com"
13    },
14    "hotel": {
15      "name": "INTERCONTINENTAL PARIS LE GRAN",
16      "rating": 4,
17      "total_price": 3399.720,
18      "price_per_night": 679.9456,
19      "currency": "USD",
20      "link": "https://www.amadeus.com"
21    },
22    "car": {
23      "company": "Budget",
24      "car_type": "SUV - Standard",
25      "price_per_day": 75.5,
26      "currency": "USD",
27      "link": "https://www.budget.com/en/vehicles/us"
28    }
29  },
30   "attractions": [
31     {
32       "created_time": "2025-12-08T03:08:13",
33       "updated_time": "2025-12-08T03:08:13",
34       "reply": "Here is your travel plan. Your trip total is quite a bit above your budget!"
35     }
36   ]
37 }
```



A screenshot of a REST client interface showing a GET request to the endpoint `[[{full_url}]]/chats/{{chat_id}}`. The response body is a JSON object containing chat details: current slots, origin/destination airport codes, city names, dates, passenger count, budget, hotel request, car request, attractions, and status.

```
1 {
2   "current_slots": {
3     "slot_id": "01KBXYVHCYXNY24Y2ERWY3A657",
4     "chat_id": "e8ab9ffb-febb-47e3-a5ce-4cacaab5fa3f",
5     "origin_airport_code": "SFO",
6     "destination_airport_code": "CDG",
7     "destination_city_name": "Paris",
8     "destination_city_code": "PAR",
9     "dates": {
10      "start": "2025-12-15",
11      "end": "2025-12-20"
12    },
13    "pax": {
14      "adults": 2,
15      "kids": 1
16    },
17    "budget": 5000.0,
18    "hotel": {
19      "request": true,
20      "amenities": [
21        "spa"
22      ],
23      "rating": 4
24    },
25    "car": true,
26    "attractions": []
27  },
28  "status": "booked"
29 }
```

Figure 14 - Get Chat by ID

/chat/{{chat_id}}: it retrieves chat history from the DB by ID for only authenticated users

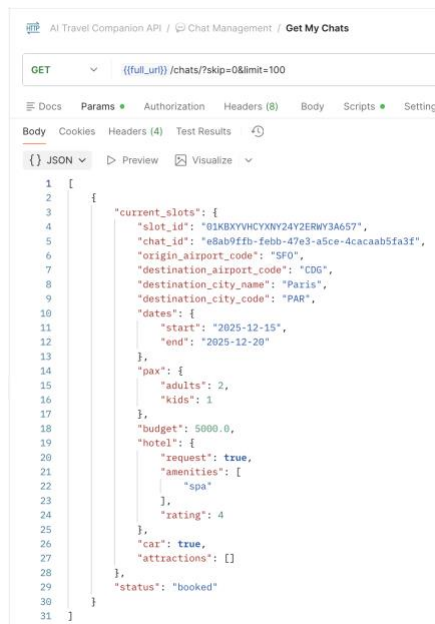


Figure 15 - Get Chats

/chat/: it retrieves all chat history from the DB for only authenticated users, with pagination of 100 chats per page

Figure 16 - Get Chat Messages

/chat/{{chat_id}}/message: retrieves individual messages and the AI bot response, plus timestamp

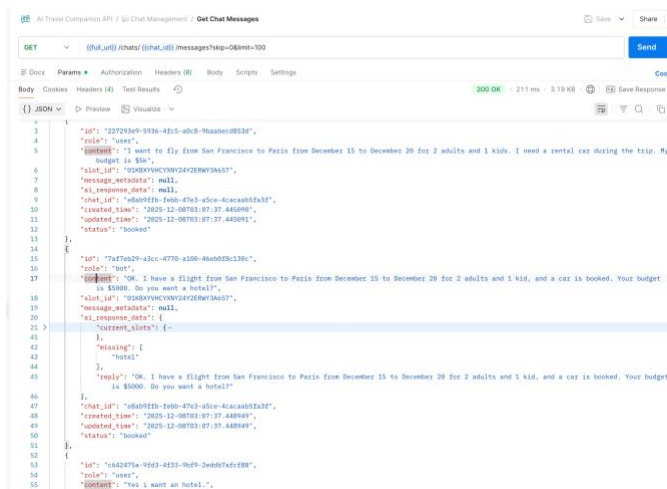
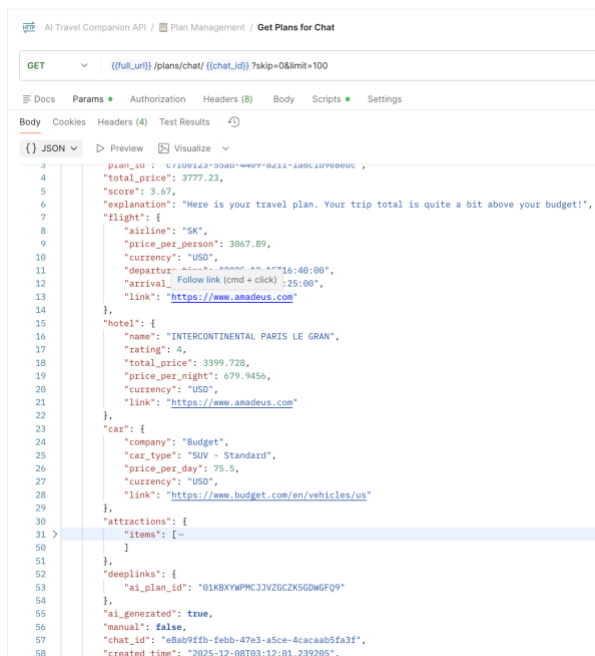


Figure 17 - Get Chat Plans

plans/chat/{{chat_id}}: retrieves individual chat plans with the chat_id or slot_id for optimized plan generation based on analyzing user behavior



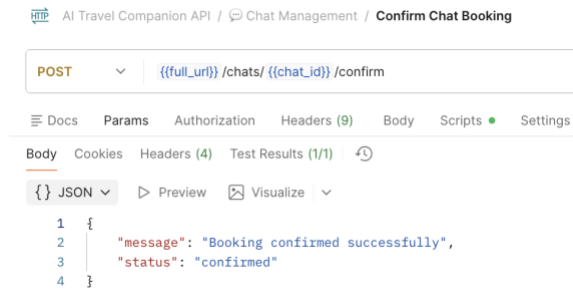


Figure 18 - Confirm Chat bookings

chats/{{chat_id}}/confirm: update chats status to confirm when flight, hotels, cars, and attraction bookings are

5. Cloud Deployment and Infrastructure (Ayo)

Containerization:

- Docker: All services are containerized using Docker for consistent deployment
- Docker Compose: Local development and production orchestration
- Multi-stage Builds: Optimized Docker images for production deployment



Figure 19 - Docker logs and services deployed

Containerized Services:

- AI Service: Fast API entry port container for AI processing (**8000**)
- Backend API: FastAPI-based RESTful service port (**8001**)
- Database: PostgreSQL 13 (s390x architecture compatible)
- Networking: Docker bridge network (ai_travel_network)

Deployment Architecture:

- LinuxONE Virtual Machines: Production deployment target (IBM Cloud or on-premises)
- Docker Hub: Container image registry for image distribution
- GitHub Actions: Continuous Integration/Continuous Deployment (CI/CD) pipeline

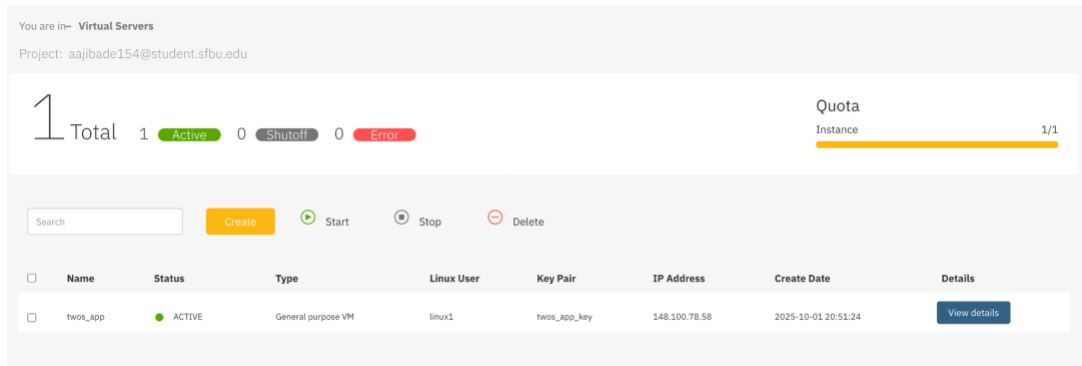


Figure 20 - IBM LinuxOne virtual server

Prerequisites:

- Docker CE for IBM Z/LinuxONE and Docker Compose
- Sufficient storage (min 20GB recommended)
- Leverage LinuxONE's vertical scaling capabilities
- Adjust CPU/Memory allocations in the compose file

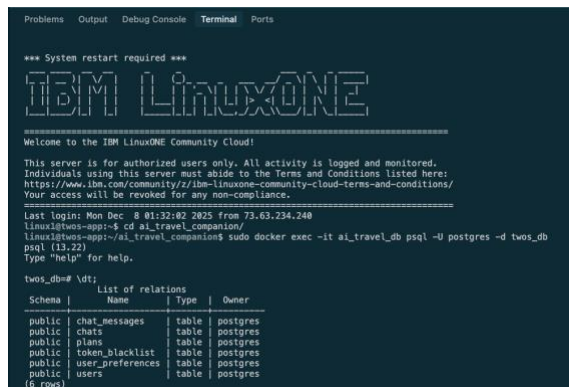


Figure 21 - IBM LinuxOne PostgreSQL virtual server

Sample Database Table

Description: Example of database tables in VM.

- Open ports: **5432** (Database)

CI/CD Pipeline:

- Build Phase:
 - Compiles and packages backend and AI service code
 - Creates optimized Docker images
 - Tags images with version metadata (branch, commit SHA, latest)
- Push Phase:
 - Pushes images to Docker Hub registry
 - Maintains image versioning and caching strategies
- Deploy Phase:
 - SSH-based deployment to target LinuxONE VMs
 - Dynamically generates production docker-compose configuration
 - Performs rolling updates with zero-downtime deployment strategies
 - Verifies service health post-deployment

Service Orchestration:

Docker Compose: Production service orchestration on target VMs

Health Checks: Automated health monitoring for all services

Dependency Management: Service startup ordering (database → backend → AI service)

Networking: Isolated Docker network for inter-service communication

Optional Infrastructure Components:

- Nginx Reverse Proxy: Optional load balancing and SSL termination (not currently in active deployment)
- Multiple VM Deployment: Support for horizontal scaling across multiple LinuxONE VMs

Scalability & Performance

1. Horizontal Scaling

- AI Service: Stateless design allows multiple instances
- Backend Service: Can be scaled independently
- Database: PostgreSQL supports read replicas for scaling reads

2. Response Time Optimization

- Immediate User Responses: Non-blocking persistence
- Parallel API Calls: Concurrent Amadeus API requests
- Efficient Database Queries: Indexed fields and optimized queries

3. Resource Management

- Connection Pooling: SQLAlchemy connection pooling
- Container Resource Limits: Docker resource constraints
- Health Monitoring: Automated health checks and restart policies

Conclusion

Completing TWOS has been a much deeper experience than we originally expected when we first pitched the idea of an AI-powered travel companion. What started out as a simple thought, something along the lines of “wouldn’t it be nice if planning a trip felt easier,” eventually grew into a full ecosystem with a frontend, NLP engine, backend services, database infrastructure and cloud deployment. Looking back at the work as a whole, it feels like each part of the system pushed us to learn something new, sometimes the hard way, but always in a direction that made the project stronger.

The frontend taught us how important small details are when designing for mobile users. The NLP module revealed how complex natural language understanding can become once real conversation patterns enter the picture. And the backend and infrastructure layers showed us the discipline required to build something stable enough to store data securely, run under load, and communicate cleanly with external services. Even the deployment stage, which we thought would be fairly straightforward, reminded us how many moving pieces a production-like environment actually involves. In a way, every layer of TWOS shaped the final product.

As a team, we also learned how challenging and rewarding collaboration can be when everyone is responsible for large, interconnected components. There were moments when things did not work, or when a change in one area unexpectedly affected another, and we had to step back and re-align our approach. But those moments honestly helped us understand the importance of clear communication and version control and, sometimes, just patience.

TWOS is not a perfect system, and we know there are many features we still want to add or improve. However, the version we have now is something we feel genuinely proud of. It runs end to end, users can chat naturally with the assistant, travel plans are generated from real data, and everything is stored, processed and displayed through an architecture that we built ourselves. It feels like more than just a capstone project. It feels like a foundation we could actually continue building on beyond the course.

Overall, this project gave us a realistic picture of what developing a real product looks like, with all its complexity, learning curves, and small victories. It challenged us technically and creatively, and it gave us a chance to apply the skills we've been studying in a way that feels meaningful. We hope TWOS can keep growing after this class, but even as it stands today, it represents a complete system that reflects our effort, our teamwork and the knowledge we gained throughout the semester.

Team Reflections

Working together on TWOS gave us a much clearer sense of what real collaborative development feels like. At first, we thought the separation of responsibilities would make things simple, but we quickly realized that every part of the system depended on something else. When the NLP structure shifted, the backend had to adjust, and when the backend changed, the frontend needed new handling. It created a kind of chain reaction that forced us to communicate far more than we expected. Honestly, it sometimes felt like we were chasing the project as it evolved, but those moments taught us how important it is to slow down, talk through decisions and not assume everyone is on the same page. There were evenings when we sat with a small bug for hours and then suddenly everything clicked, and the sense of relief almost made the struggle worth it.

As the project continued, we began to understand each other's working styles in a more realistic way. Some of us plan extensively and think through every step, while others experiment and figure things out as they go. Instead of clashing, the mix somehow became a strength. It created a rhythm where one person's structure balanced another's spontaneity. We noticed that even when the work was difficult or felt messy, we were still learning from one another. And by the time all the parts finally connected into a functioning system, it felt like a genuine team accomplishment. TWOS became more than a course project. It became a shared experience that reminded us that building something meaningful often involves uncertainty, trial and error and a willingness to adapt together.

References

Booking.com Demand API. (2025). Booking.com.
<https://developers.booking.com/demand/docs/open-api/demand-api>

Connect to Amadeus travel APIs | Amadeus for Developers. (n.d.). Amadeus IT Group SA.
<https://developers.amadeus.com/self-service>

Flights - Expedia Group Developer Hub. (2025). Expediagroup.com.
<https://developers.expediagroup.com/rapid/flights#step-1---search-available-flight-offers>

<https://dev.to/gbengelebs/netflix-system-design-backend-architecture-10i3>

Swagger-API. (n.d.). GitHub - swagger-api/swagger-ui: Swagger UI is a collection of HTML, JavaScript, and CSS assets that dynamically generate beautiful documentation from a Swagger-compliant API. GitHub. <https://github.com/swagger-api/swagger-ui>

The PostgreSQL Global Development Group. (2025). PostgreSQL: Documentation. Www.postgresql.org. <https://www.postgresql.org/docs/>

Welcome to the Tiqets Distributor API | Tiqets for developers. (2025, September 5). Tiqets.dev.
<https://developers.tiqets.dev>

https://dbdiagram.io/d/AI_Travel_Companion-68be1d8461a46d388ede1ce4

draw.io

<https://github.com/linuxone-community-cloud/technical-resources/blob/master/faststart/deploy-virtual-server.md>